Ports and sockets

which app gets the packet

Watch the 9-min video

In one line: the IP address gets a packet to the device, and the port, a 16-bit number from 0 to 65,535, gets it to one app on that device; servers listen on well-known ports such as 443, clients are lent a random high port, and a connection is a pair of sockets: four values that make it unique.

WORDS YOU'LL MEET TODAY

Port: a number (0 to 65,535) that names one app on a device

Listening: an app waiting for packets on its port

Well-known port: a fixed number for a common service, 0 to 1023 (443 = HTTPS)

Connection: a pair of sockets: client IP and port, server IP and port

Socket: one end of a conversation: an IP address plus a port

Protocol: agreed rules for talking; TCP and UDP are the two main ones

Ephemeral port: a high random port lent to a client for one conversation

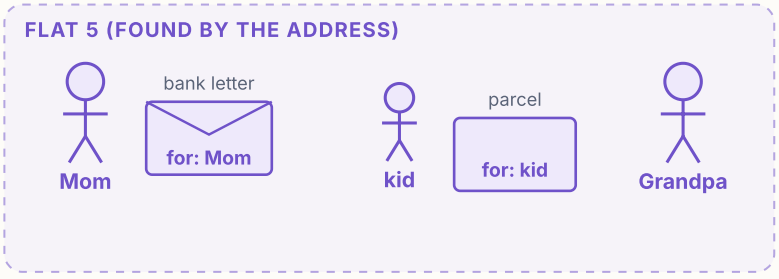
Address already in use: the error when a second app tries to listen on a taken port

1 The question: which app gets the packet?

Your phone is running WhatsApp, Chrome and Spotify, all at the same time. A message arrives, a web page arrives, a song arrives. But in lesson 3 we saw that your phone has one IP address, and all three apps share it. So when a packet arrives, how does the phone know which app it's for?

2 An everyday picture first: the people inside the flat

In lesson 3, the address got the courier to one flat. But a flat has people in it: Mom, the kid and Grandpa. The courier brings a bank letter and a parcel. The flat number gets both to the door; the **name** on each one says who inside wants it. The letter is for Mom, the parcel is for the kid. And someone must be home, waiting for it: if nobody with that name lives there, the letter can't be delivered.



The flat number gets the post to the door; the name says who inside wants it. No such person: it can't be delivered.

Figure 1. The flat (lesson 3's device) has people inside: the apps. The name on each envelope is the port, so the right person gets it.

3 Now the real thing

The flat	In a network
The flat	your device, found by its **IP address** (lesson 3)
The people inside	the **apps** running on it
The name on the envelope	the **port number**, written in each packet's header
Someone at home, waiting	an app **listening** on its port

So every packet carries two things: the IP address, to reach the device, and the port, to reach one app on it. Together they say: this device, this app.

4 What a port looks like

A port is just a number, written after the address with a colon: **192.168.1.5:443** means the device 192.168.1.5, port 443. A port is stored in 16 bits, and 2^{16} = 65,536, so ports go from **0 to 65,535**: over 65,000 possible ports on every device.

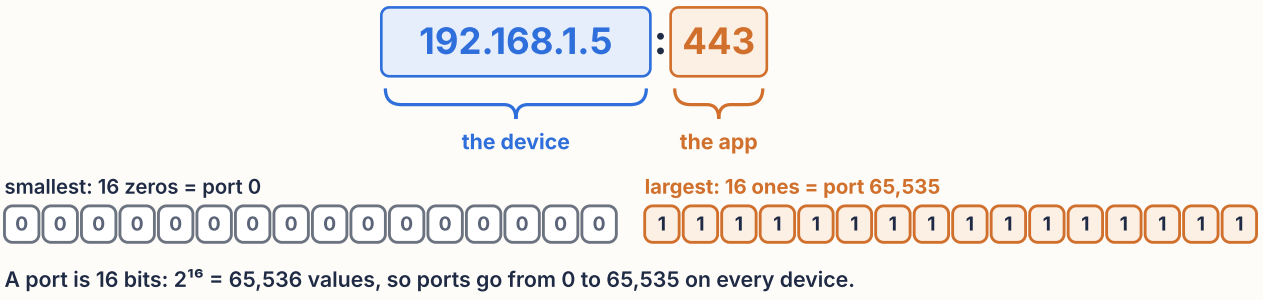


Figure 2. Anatomy of 192.168.1.5:443: the IP address finds the device, the port after the colon finds the app. A port is 16 bits, 0 to 65,535.

The low ones, **0 to 1023**, are **well-known ports**, fixed for common services:

Port	Service	What it's for
80	HTTP	the web
443	HTTPS	the secure web
22	SSH	logging in to a server
53	DNS	looking up names (lessons 15 to 17)
25	SMTP	sending email between mail servers

That's why you never type a port: open a web address, and the browser uses 443 by itself (`https://backendroadmap.com` means `backendroadmap.com:443`).

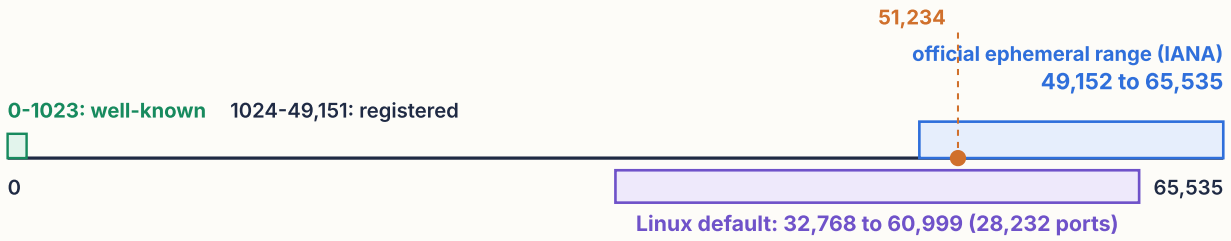
5 Listening, and the protocol

On a server, an app opens a port and waits. That's called **listening**, like Mom at home, waiting for her letter. A web server listens on 443, and a packet for port 443 goes to it, and only to it. One more detail: each port belongs to a **protocol**, TCP or UDP, two sets of rules for sending data. Port 53 over TCP and port 53 over UDP are separate. TCP is lessons 8 to 11, and UDP is lesson 12.

6 Your side: an ephemeral port

Your browser isn't a well-known service. So when it connects, your laptop lends it a port: a high random number, for example **51,234**. It's called an **ephemeral port**, because it lasts only as long as the conversation.

The official range is **49,152 to 65,535**; Linux uses **32,768 to 60,999** by default (28,232 ports).



Servers listen on well-known ports. Clients are lent a high random (ephemeral) port, such as 51,234, for one conversation.

Figure 3. All 65,536 ports on one line (to scale): well-known 0 to 1023 at the very start, the official ephemeral range 49,152 to 65,535, and the Linux default 32,768 to 60,999.

7 Sockets and connections

A **socket** is one end of a conversation: an IP address plus a port. Your laptop's end is **203.0.113.7:51234**. The server's end is **198.51.100.10:443**. A **connection** is the pair of sockets: four values, client IP, client port, server IP, server port. No two connections share all four.

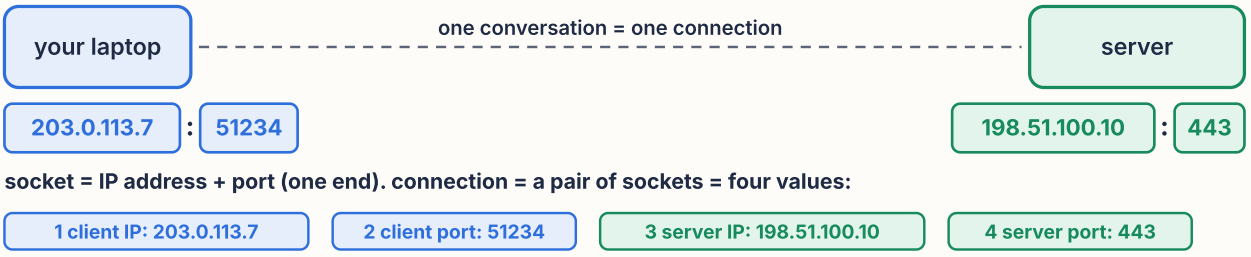


Figure 4. A socket is one end of a conversation, an IP address plus a port. A connection is the pair of sockets: four values.

Connection	Client IP	Client port	Server IP	Server port
your tab 1	203.0.113.7	51234	198.51.100.10	443
your tab 2	203.0.113.7	**51235**	198.51.100.10	443
a friend	**192.0.2.25**	50112	198.51.100.10	443

So one server port can serve many people: same server port, but every connection is unique.

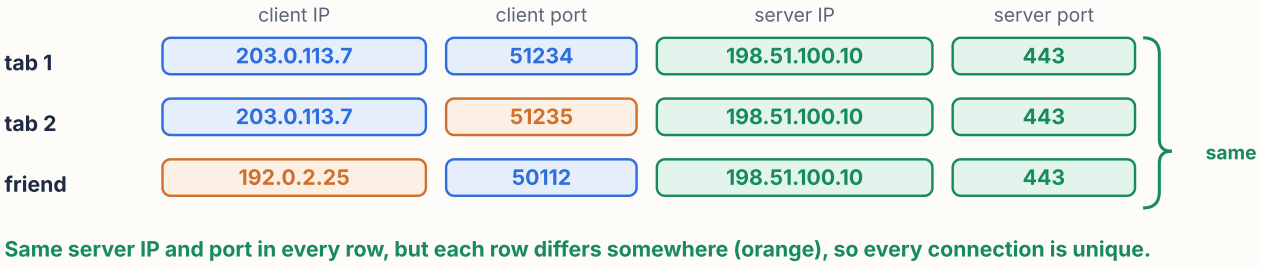
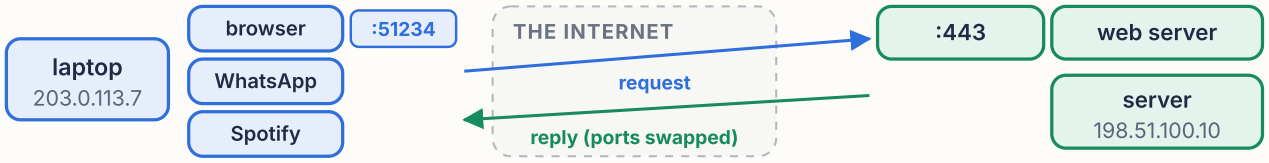


Figure 5. One server port serves many connections: a second tab gets another client port, a friend has another address, and every four-value row is still unique.

8 Step by step: one request

1. **Listen.** The web server starts and listens on 443. Job: be ready before anyone calls.
2. **Get a port.** Your browser asks for a socket and gets port 51,234. Job: give the reply a way back to this one app.

3. **Send.** The packet goes out from 203.0.113.7:51234 to 198.51.100.10:443. The server reads the port and hands it to the web server. Job: carry both ends, so each side knows where to reply.
4. **Reply.** The reply comes back with the ports swapped: from 443, to 51,234. Your laptop reads the port and gives it to the browser, not to WhatsApp, not to Spotify. Job: the right app, every time.



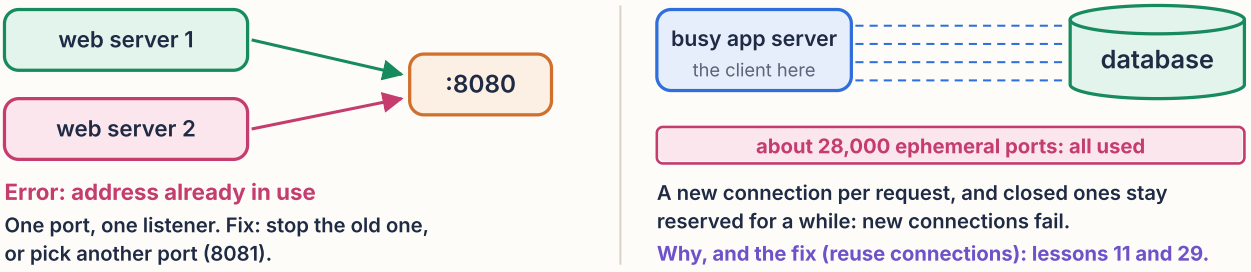
1 Listen. The web server starts and listens on 443. Job: be ready before anyone calls.
2 Get a port. The browser asks for a socket and is lent port 51234. Job: give the reply a way back to this one app.
3 Send. From 203.0.113.7:51234 to 198.51.100.10:443; the server reads port 443 and hands it to the web server.
4 Reply. From 198.51.100.10:443 to 203.0.113.7:51234; the laptop reads 51234 and gives it to the browser, not WhatsApp.

Figure 6. One request, step by step: listen, get a port, send, reply. The ports in the header pick the app at each end.

For the curious: at home, your router also rewrites the port as it shares one public address, so the server may see a different client port. That's NAT, lesson 14. 203.0.113.7, 198.51.100.10 and 192.0.2.25 come from the ranges reserved for documentation (RFC 5737), so they belong to nobody real.

9 What goes wrong

- **Port already in use.** You start a web server on port 8080, then a second one, also on 8080. It fails: `address already in use`. One port, one listener; otherwise the system couldn't know which app gets the packet. The fix: stop the old one, or pick another port.
- **Running out of ports.** Linux has about 28,000 ephemeral ports. A busy server that opens a new connection to its database for every request can use them all, because closed ones stay reserved for a while. New connections fail. Why, and the fix, is lesson 11 (and reusing connections, lesson 29).



Error: address already in use
One port, one listener. Fix: stop the old one, or pick another port (8081).

A new connection per request, and closed ones stay reserved for a while: new connections fail.
Why, and the fix (reuse connections): lessons 11 and 29.

Both failures are about the same rule: each socket on a device must be unique.

Figure 7. What goes wrong: two apps trying to listen on one port (address already in use), and a busy client using up its ephemeral ports.

PAUSE AND THINK

A shop's server listens on port 443. 100,000 phones connect at the same time. Does it need 100,000 ports? Cover the answer and decide.

Answer. No. They all use the same server port, 443. Each connection is still unique, because the client's address or port is different. One port, 100,000 different pairs.

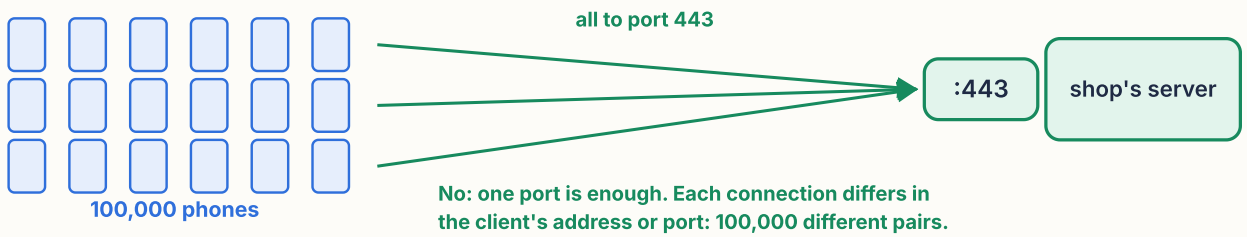


Figure 8. The pause-and-think answer: 100,000 phones can all connect to port 443; each four-value pair is different, so one server port is enough.

TWO COMMON MISTAKES

- **"A server needs a new port for every client."** No. Every client connects to the same listening port; the four values keep the connections apart.
- **"A port and a socket are the same thing."** No. A port is just a number (0 to 65,535). A socket is an IP address plus a port: one end of a conversation.

RECAP IN THREE LINES

1. The IP address finds the device, and the port, a number from 0 to 65,535, finds the app.
2. Servers listen on well-known ports, like 443, and clients get a random high port.
3. A socket is an IP address plus a port, and a connection is a pair of sockets: four values.

10 Interview questions: what to say in 30 seconds

Q1. What is a port, and why do we need one if we already have an IP address? BASIC

A port is a 16-bit number, 0 to 65,535, that identifies one app (one service or one conversation) on a host. The IP address only gets a packet to the device; many apps share that one address, so the port in the packet's header tells the device which app gets it.

Q2. What is a socket, and what identifies a connection? BASIC

A socket is one end of a conversation: an IP address plus a port, such as 203.0.113.7:51234. A connection is a pair of sockets, so it is identified by four values: client IP, client port, server IP and server port (plus the protocol, TCP or UDP). No two live connections share all of them.

Q3. What are well-known ports and ephemeral ports? BASIC

Well-known ports, 0 to 1023, are fixed for common services: 80 HTTP, 443 HTTPS, 22 SSH, 53 DNS, 25 SMTP. Servers listen on them so clients know where to connect. A client is lent an ephemeral port, a high random number, for one conversation: the official range is 49,152 to 65,535, and Linux uses 32,768 to 60,999 by default.

Q4. You start your app and get "address already in use". What happened, and what do you do? SCENARIO

Another process is already listening on that port, maybe an old copy of your app. Only one app can listen on a given IP address and port, otherwise the system could not tell which one gets the packet. Find and stop the old process, or start your app on a different port.

Q5. How can one server port, 443, serve 100,000 clients at once? TRICKY

The server does not need a port per client. Each connection is the four values, and while the server IP and port are the same for all of them, the client IP or client port is different in every one, so each connection is still unique. The limits are memory, file handles and CPU, not the port number.

Q6. Can a client run out of ports? TRICKY

Yes. A client has only a few tens of thousands of ephemeral ports (about 28,000 on Linux by default) for connections to one server address and port. A busy service that opens a new connection for every request, while closed ones stay reserved for a while (TIME_WAIT), can use them all and new connections fail. The fixes, such as reusing connections, are in lessons 11 and 29.

Read more. RFC 793 / RFC 9293 (TCP), RFC 768 (UDP), RFC 6335 (port number ranges and the IANA registry), RFC 6056 (choosing ephemeral ports), RFC 5737 (example addresses). The full written lesson is at backendroadmap.com.

Next: SD-N-005 · Protocols and layers: agreed languages, and why we stack them backendroadmap.com